

AI PROGRAMMING · PROJECT WORK · SUMMER SEMESTER 2026

Intelligent Autonomous Navigation System

*Combining global search, local optimization, and adaptation
into one intelligent robot controller*

Obaidullah Habibi

Matriculation No. 807634 · Study course: AINB

Supervisor: Prof. Dr. Isabel Hübener

PROBLEM DESCRIPTION

What are we actually solving?

Move a mobile robot from a start to a goal in a **partially known** environment — reaching it **safely** and **efficiently**.



The world is only partially known

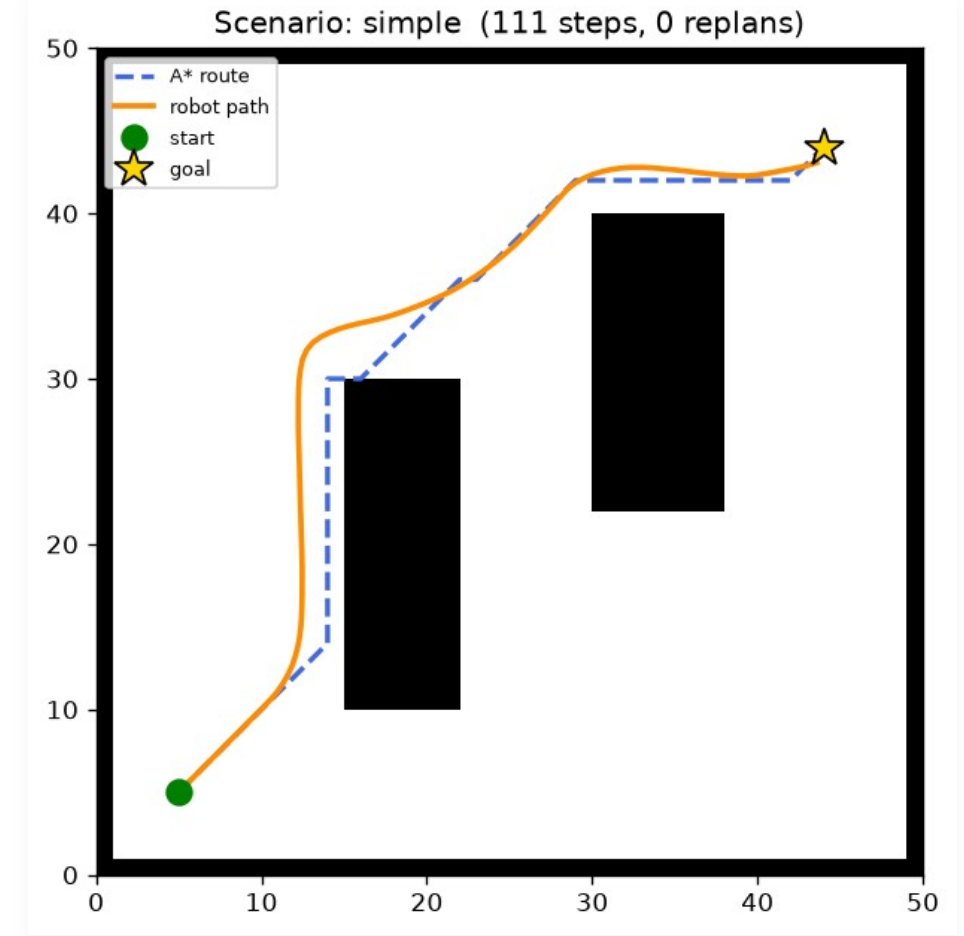
Static walls are on the map. Moving obstacles are NOT — they can only be handled by reacting at run time.



The robot is physically constrained

It cannot teleport. Bounded speed, turn rate and acceleration mean not every motion is reachable.

This is why it is a genuine AI problem — not just drawing a line on a map.



A concrete instance: start (green) → goal (star), planned route vs. the path actually driven.

Why no single algorithm is enough



Long-horizon reasoning

Guarantees a route exists and avoids dead ends.

Needs to look at the whole map at once.

Provided by the GLOBAL planner



Short-horizon reactive reasoning

Respects the robot's dynamics.

Dodges obstacles that appear only while driving — reasons about the next fraction of a second.

Provided by the LOCAL planner

No single layer provides both. The system combines a global planner + a local optimizer + replanning so each layer covers the other's weakness.

A two-layer control hierarchy

The code is split into small, single-responsibility modules, making the two-layer structure explicit.

1

Once, at the start

The global planner computes an optimal, collision-free route over the static map.

2

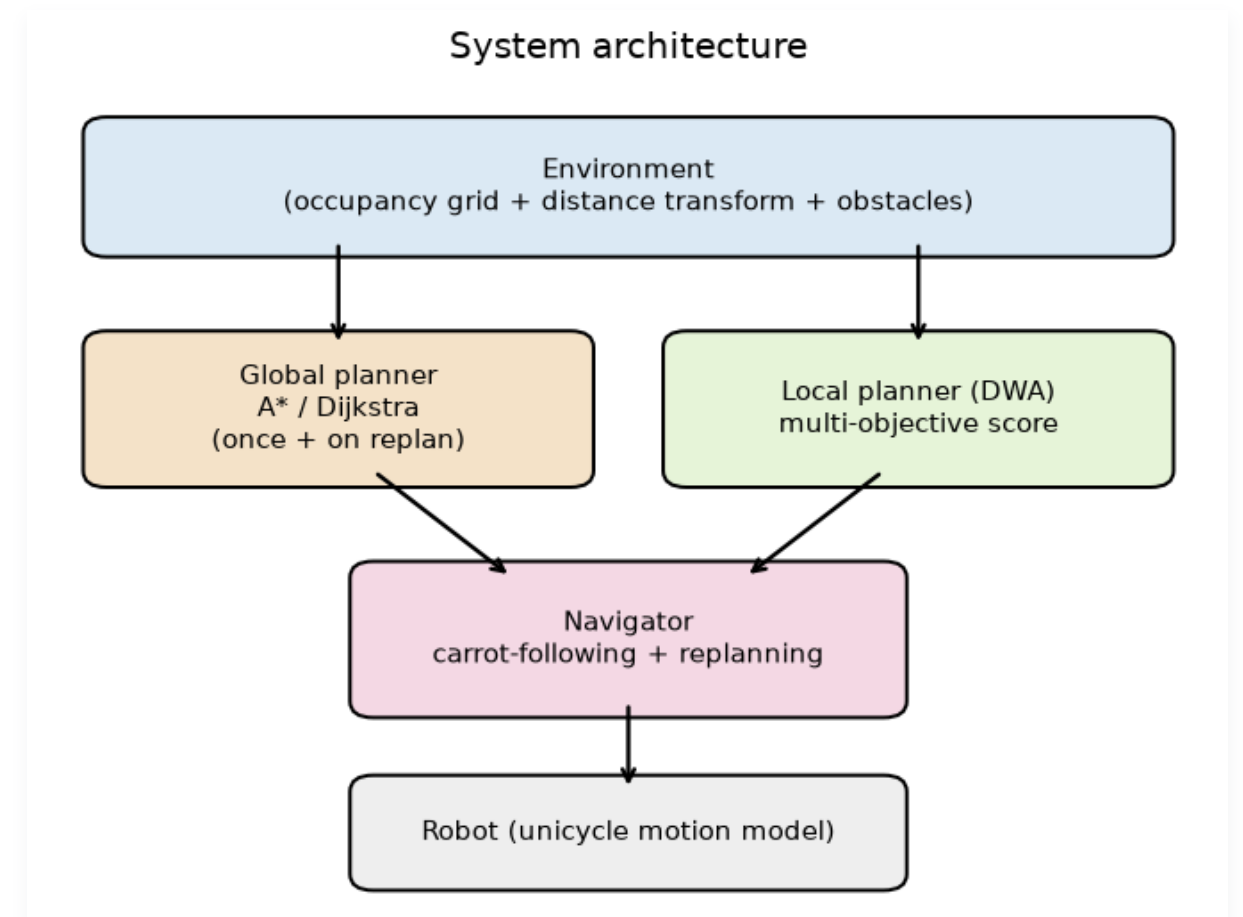
Every control step

The local planner picks a carrot point ahead on the route and chooses a feasible motion, reacting to moving obstacles.

3

When fully blocked

The navigator replans globally from the robot's current cell — this closes the loop for adaptation.



Environment feeds both planners → navigator drives the robot → replans when stuck.

One grid, two uses — plus a speed trick

A 50×50 occupancy grid. $\text{grid}[\text{row}, \text{col}] = 1$ (occupied) or 0 (free). The same map serves both the discrete global search and the continuous local motion.

Static obstacles

Baked into the grid — the known world.

Dynamic obstacles

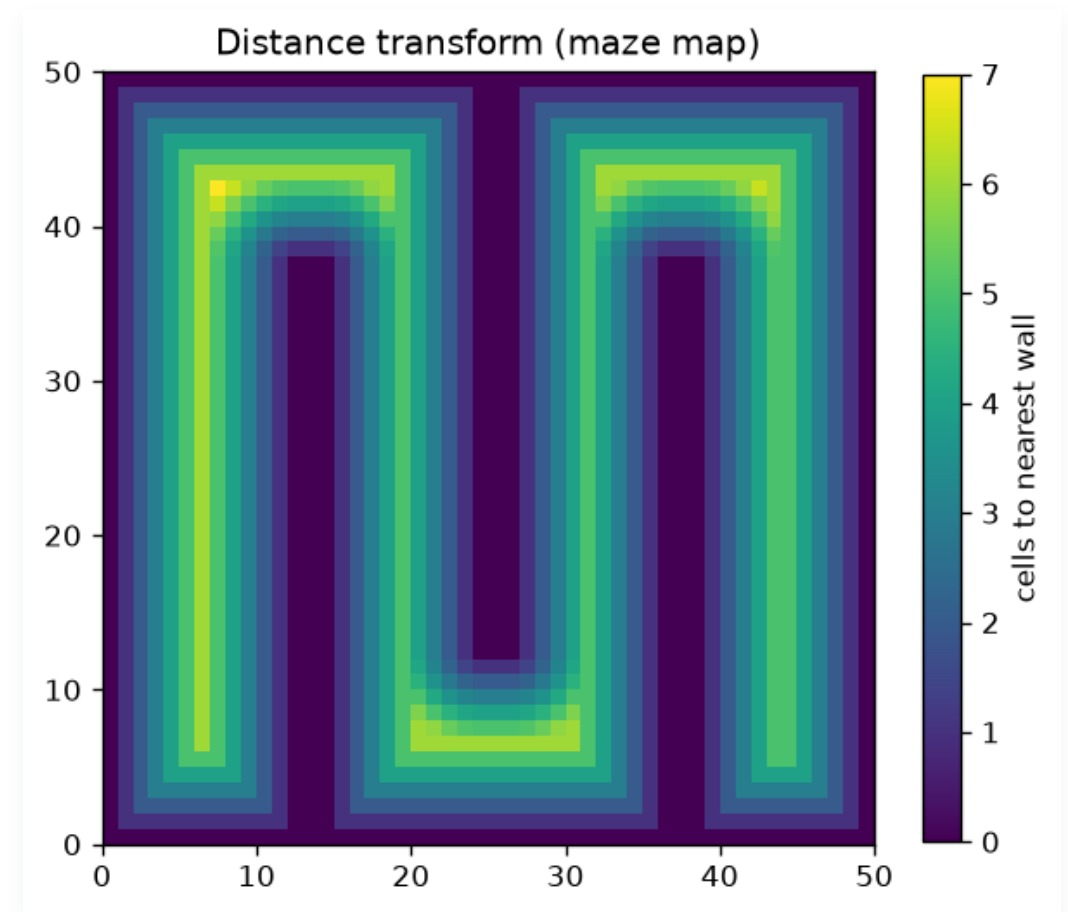
Moving circles NOT in the grid — the unknown world.

The distance transform — the key speed trick

Precomputed once per map: for every free cell, the true Euclidean distance to the nearest wall.

$\text{clearance}(x, y) = \text{DT}[y, x] \rightarrow \text{O}(1) \text{ lookup}$

Thousands of nearest-wall queries per second become a single lookup — this is what makes real time possible.



Euclidean distance transform (maze). Bright = far from walls, dark = adjacent.

A* vs. Dijkstra — from scratch

Both share one graph-search skeleton over an 8-connected grid (*straight* = 1, *diagonal* = $\sqrt{2}$, *no corner-cutting*). Only the frontier priority differs:

Dijkstra

$$\text{priority} = g(n)$$

Expands uniformly — like a flood fill.

A*

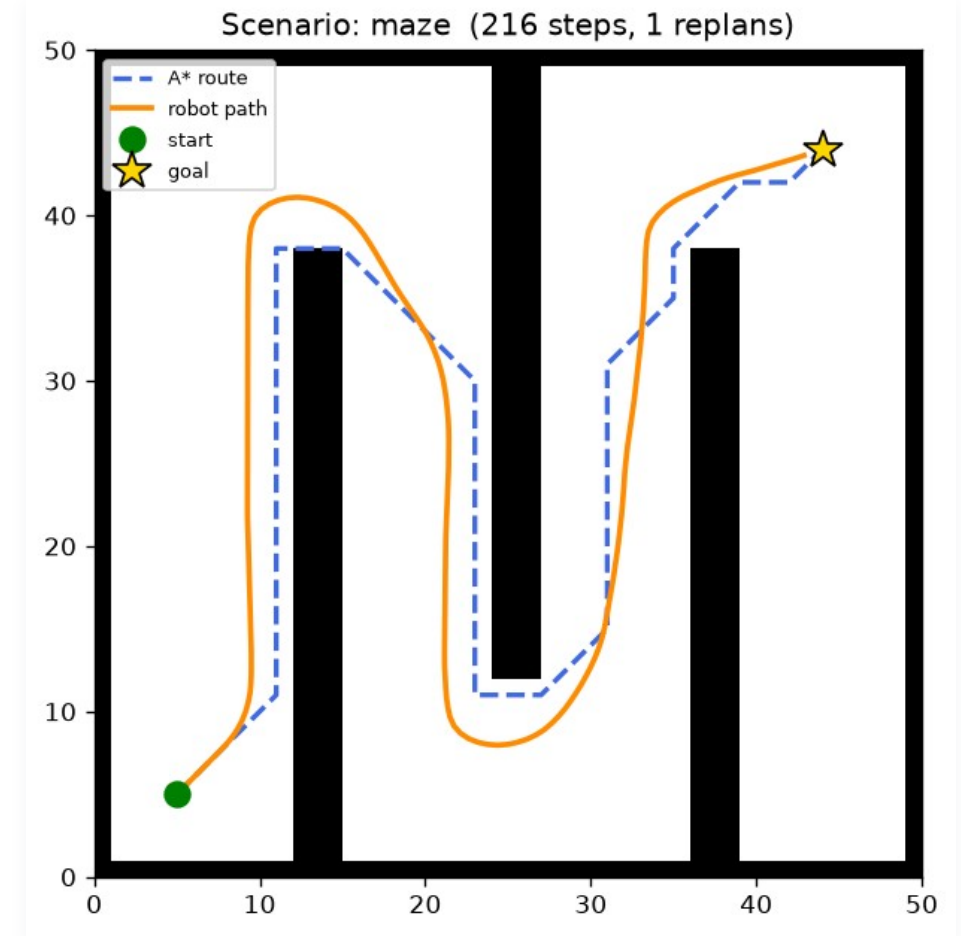
$$\text{priority} = g(n) + h(n)$$

Guided toward the goal by a heuristic.

Octile heuristic (admissible):

$$h(n) = (dx + dy) + (\sqrt{2} - 2) \cdot \min(dx, dy)$$

Because the heuristic never overestimates, A* is **guaranteed optimal** — same cost as Dijkstra, far fewer cells explored. In short: **A* is Dijkstra guided by a heuristic.**



Maze: the robot follows the global route through the corridors.

The unicycle — why the problem is non-trivial

A standard model for a wheeled robot that moves forward and rotates. State:

 x, y

position

 θ

heading

 v

forward speed

 ω

turn rate

Integration over a small step dt :

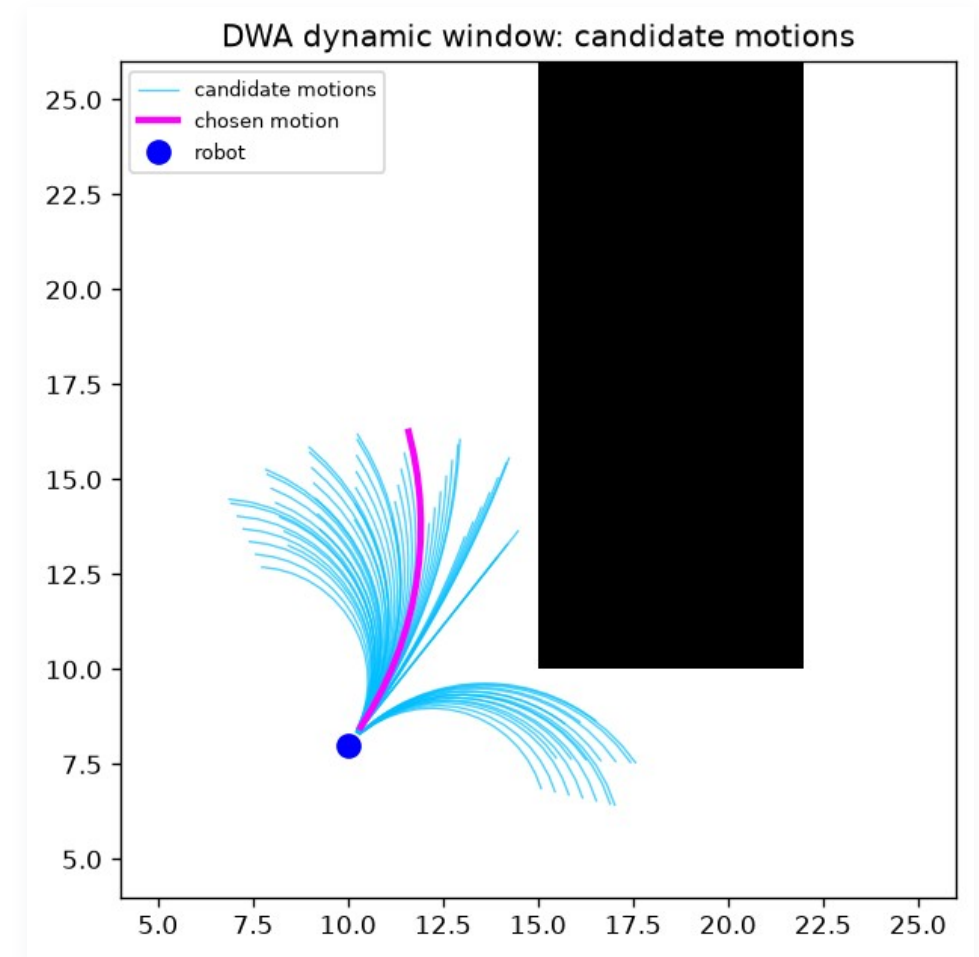
$$\theta' = \theta + \omega \cdot dt$$

$$x' = x + v \cdot \cos(\theta') \cdot dt$$

$$y' = y + v \cdot \sin(\theta') \cdot dt$$

Bounded limits

Max speed · max turn rate · max acceleration · max change of turn rate · radius · dt . The robot cannot instantly reach any velocity — so the planner may only ever pick physically reachable motions.



The motions physically reachable from the current state (preview of DWA).

The Dynamic Window Approach (DWA)

1

Build the dynamic window

The set of (v, ω) commands reachable next step given current velocity + acceleration limits. Guarantees feasibility.

2

Roll out & collision-check

Simulate each candidate ~ 1.5 s ahead with the motion model; discard any trajectory that hits a static or dynamic obstacle.

3

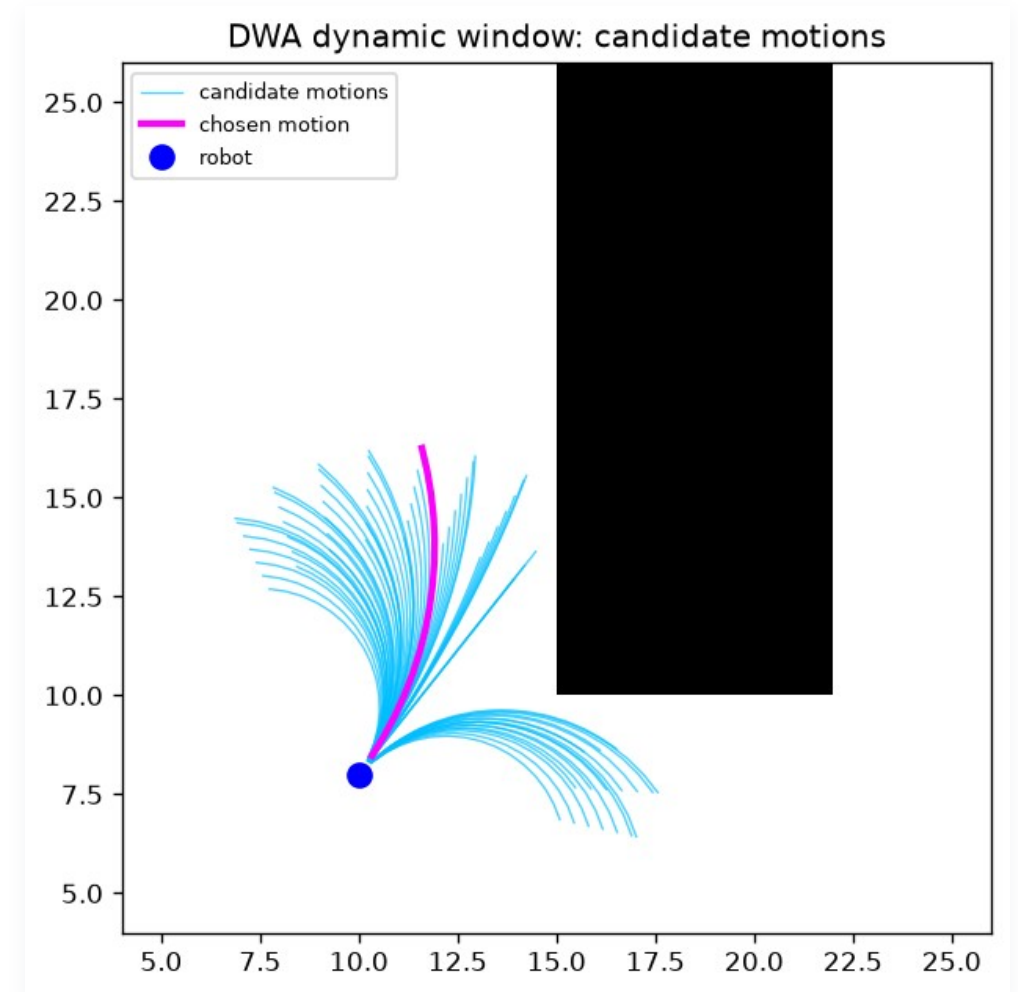
Score & execute best

Score every surviving trajectory and drive the winner.

Base score (assignment):

$$G(v, \omega) = \alpha \cdot \text{heading} + \beta \cdot \text{obstacleDist} + \gamma \cdot \text{velocity}$$

Robot follows the global route by aiming at a carrot: a look-ahead point a few cells ahead on the path.



Blue fan = reachable motions; magenta = the chosen best trajectory bending away from the wall.

Multi-objective optimization

$$G = \alpha \cdot \text{heading} + \beta \cdot \text{obstacleDist} + \gamma \cdot \text{velocity} + \delta \cdot \text{clearance} + \epsilon \cdot \text{smoothness}$$

$\delta \cdot \text{clearance}$ (safety)

Prefer a larger average margin along the whole horizon, not just at the end.

$\epsilon \cdot \text{smoothness}$ (comfort)

Penalise sharp turning & sudden changes in turn rate → smoother, more efficient motion.

Two design choices worth defending:

- Normalisation.** All five terms scaled to [0,1] before weighting, so weights are directly comparable — "safety twice as important as speed" is meaningful.
- Clipped clearance.** Distances clipped at a safe threshold, so the robot won't freeze in open space just to maximise its margin.

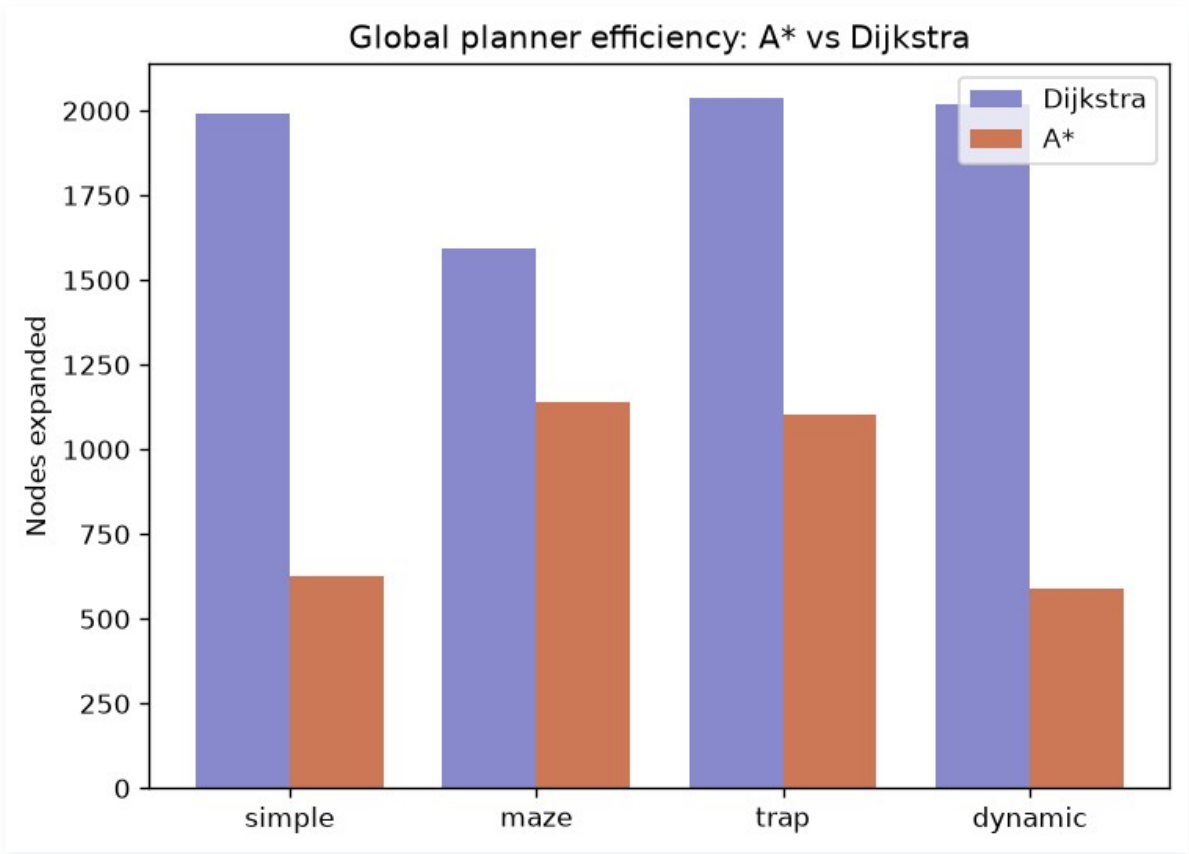
Four presets expose the trade-off (same map, just different weights):

Preset	α	β	γ	δ	ϵ
balanced	1.0	1.0	0.7	0.6	0.4
aggressive	1.4	0.5	1.4	0.2	0.2
cautious	0.8	1.6	0.4	1.5	0.6
smooth	1.1	0.9	0.7	0.5	0.9

Personality: *balanced* • *aggressive* (fast, more risk) • *cautious* (large margins) • *smooth* (gentle turns).

EXPERIMENT 1

Global planner efficiency: A* vs. Dijkstra



Nodes expanded per scenario — same optimal path cost, fewer expansions with A*.

Nodes expanded (both return the same optimal cost):

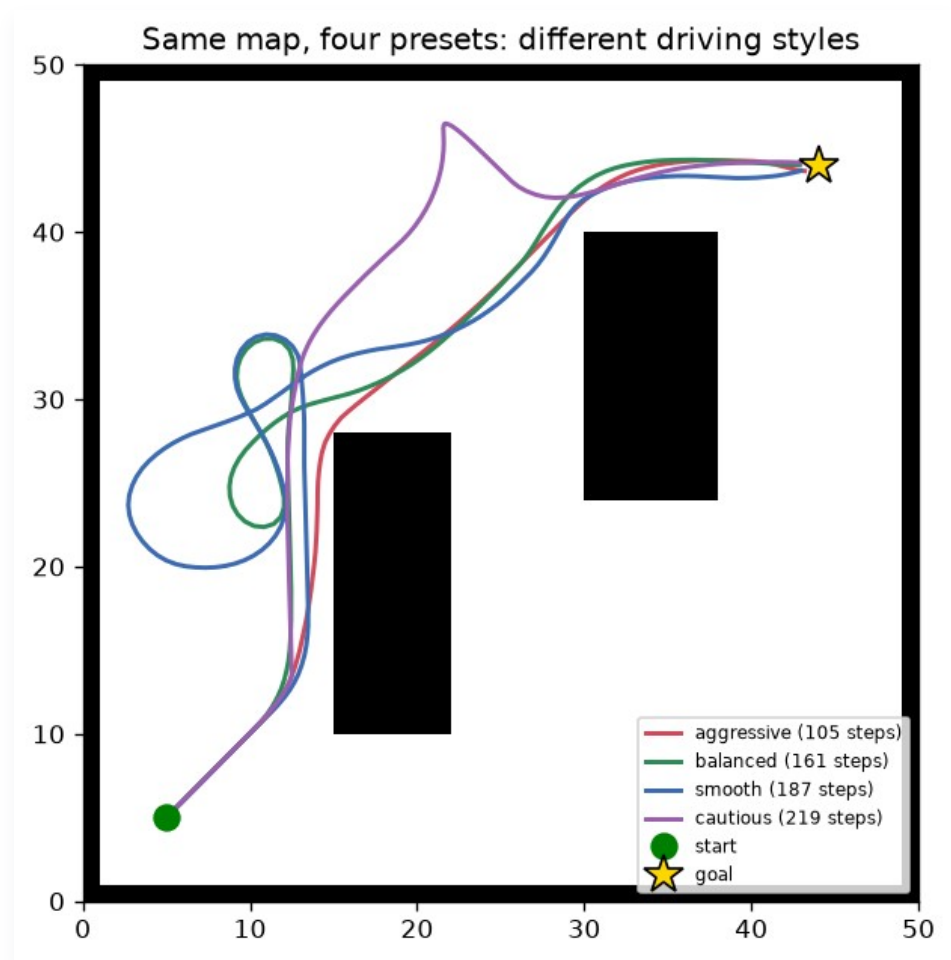
Scenario	Dijkstra	A*	Path cost
simple	1989	628	64.53
maze	1592	1139	115.60
trap	2036	1101	68.63
dynamic	2019	588	63.36

Honest, interesting result

A* wins big in open maps. In the cluttered maze the straight-line heuristic is repeatedly blocked by walls, so its advantage shrinks most — **the heuristic helps least exactly where the map is most confusing.**

EXPERIMENT 2

Weight study — four personalities, one map



Same map & obstacles, four presets — four clearly different driving styles.

Measured on the dynamic map:

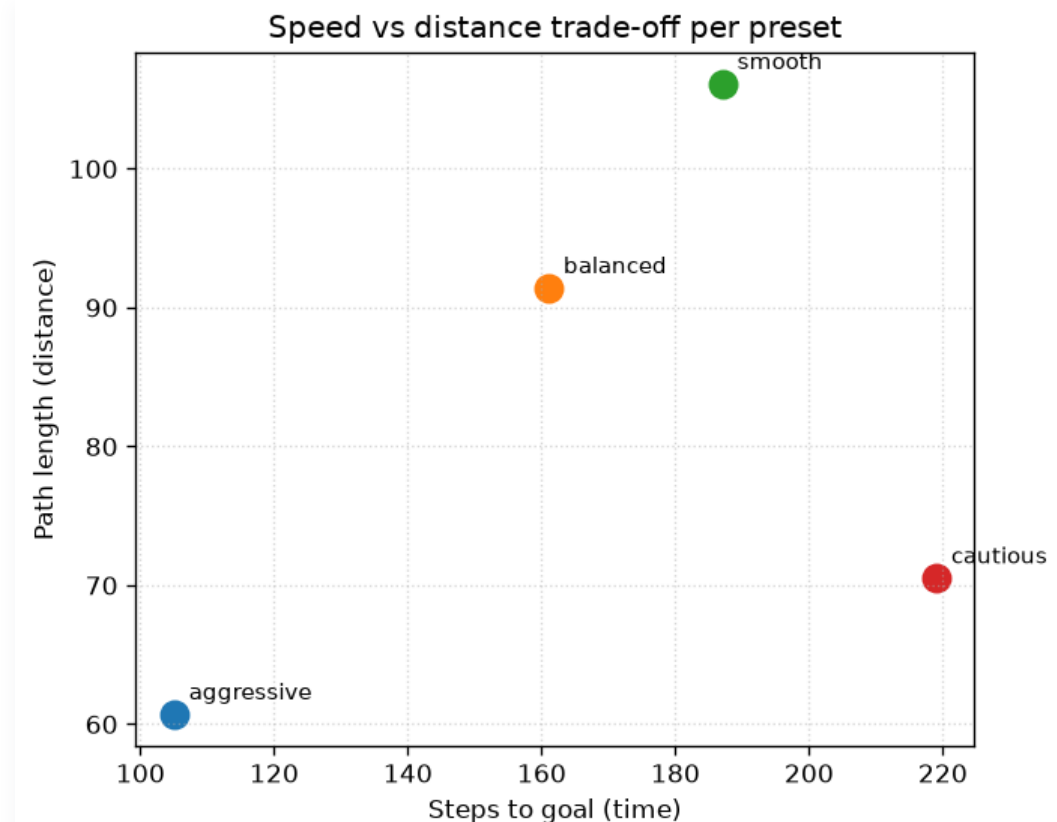
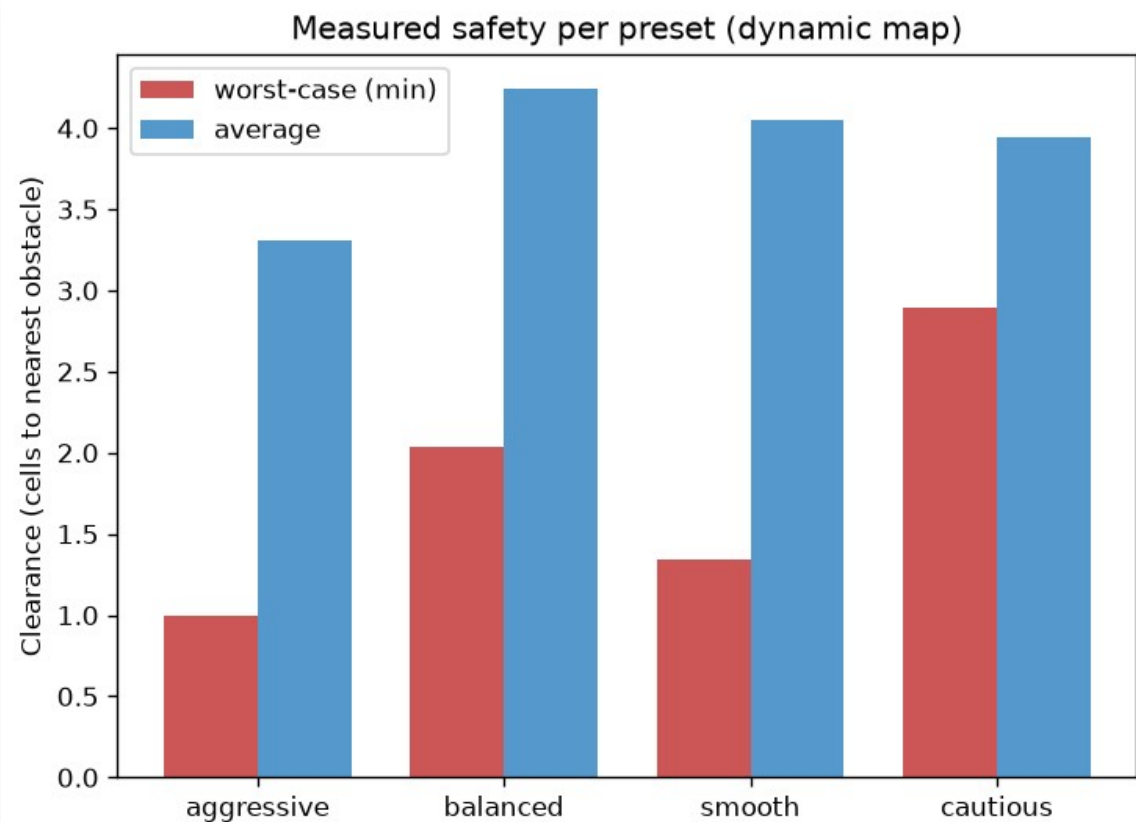
Preset	Steps	Length	Replans	Min clr	Avg clr
aggressive	105	60.7	0	1.00	3.31
balanced	161	91.4	0	2.04	4.24
smooth	187	106.1	1	1.34	4.05
cautious	219	70.5	0	2.89	3.94

The weights alone produce the behaviour

Aggressive is fastest (105 steps) but rides closest to obstacles. **Cautious & smooth** take ~2× the steps around the moving obstacles — one even had to replan.

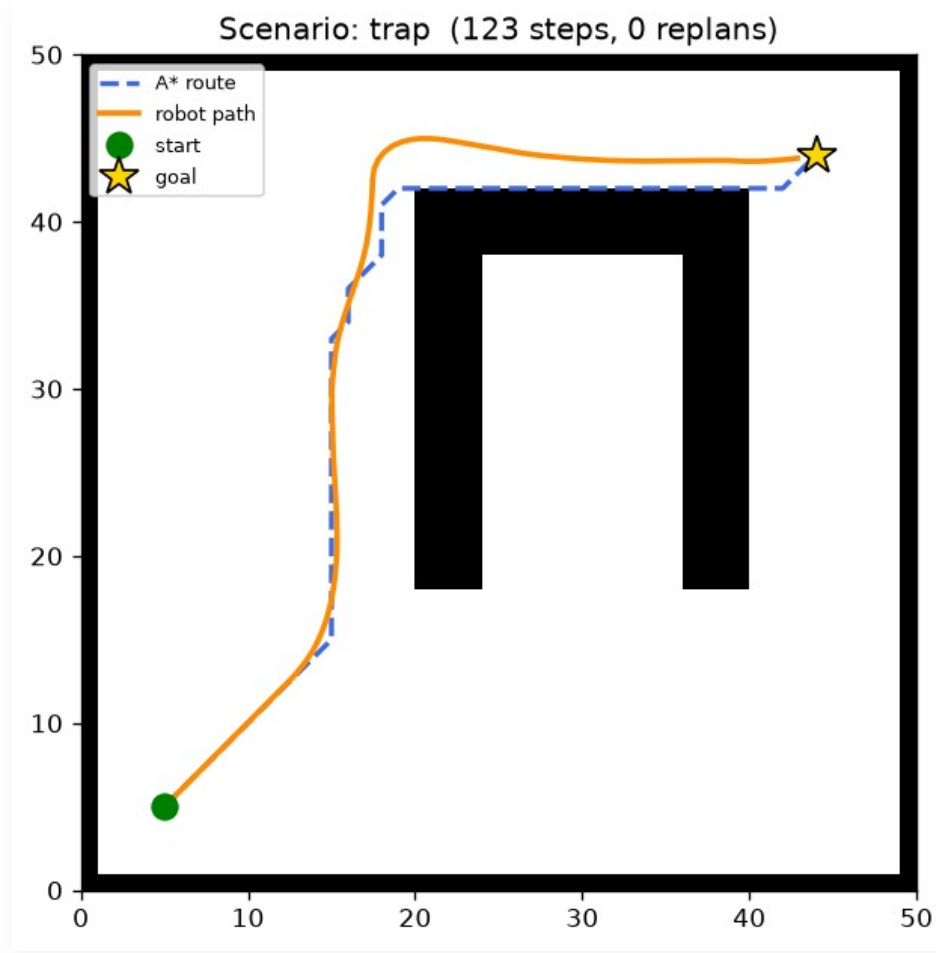
EXPERIMENT 2

The safety / speed trade-off is real



Averages look similar (clearance is clipped once "safe enough"), so the weights show their effect in the **worst case** — which is exactly what matters for safety. Aggressive wins time & distance; cautious deliberately trades time for margin.

Where — and why — it can struggle



Trap: global route (blue) goes around the U; a local-only robot drives in and gets stuck.

- **Local-only fails in the trap.** Escaping the U means temporarily moving away from the goal — a greedy driver never chooses that. The global plan routes around it in advance. The clearest proof both layers are needed.
- **A* is not always cheap.** In the cluttered maze the straight-line heuristic is repeatedly blocked, so A*'s edge over Dijkstra is smallest.
- **Safety margin is bounded.** DWA is reactive — it checks obstacles where they are now, not where they're heading — so even cautious tops out around ~2.9 cells of clearance.
- **Other limits.** Grid resolution caps path precision; the distance transform assumes static obstacles don't move.

REFLECTION

What makes the system intelligent?

Intelligence comes from no single algorithm — it emerges from four ingredients working together.



Search

Global A* / Dijkstra

Long-horizon optimality; guarantees dead ends are avoided.



Optimization

Multi-objective DWA

Real-time decisions balancing competing goals under motion limits.



Representation

Grid + distance transform

Makes both the search and the fast obstacle reasoning possible.



Adaptation

Carrot-following + replanning

Responds to a changing world it did not fully know in advance.

Summary & future work

What was built

- A complete, modular navigation system.
- A* / Dijkstra global planner — implemented from scratch.
- A Dynamic Window Approach local driver.
- A five-term multi-objective extension (Option D).
- Quantitative evaluation across four purpose-built scenarios.

Results confirm the expected (A preserves optimality, fewer nodes) and the subtle (its edge shrinks in mazes), plus a clear speed-safety-smoothness trade-off with no single best setting.*

Future work

1

Motion prediction

Plan against where each dynamic obstacle is heading, not only where it is now — the main fix for the bounded safety margin.

2

Continuous planning

Replace the grid with a sampling-based planner (RRT*) for finer, resolution-independent paths.

3

Learned risk maps

Learn from experience which regions tend to be dangerous and bias the cost function.

LIVE DEMONSTRATION

Let's watch it navigate.

Successful navigation · real-time obstacle avoidance · behaviour under different presets and scenarios

Thank you — questions welcome.

Obaidullah Habibi · 807634 · AINB

